

Clase 130 — Ingeniería inversa: introducción

Parte: **5 — Explotación de sistemas y binarios** · Fuente: *Andriesse, Practical Binary Analysis · Eilam, Reversing* 🕒 Duración estimada: **110 min** · Nivel: **Intermedio**

🎯 Objetivo

Sentar las bases de la **ingeniería inversa** de software: qué es, para qué sirve (análisis de malware, descubrimiento de vulnerabilidades, interoperabilidad, CTF), y qué contiene un binario ejecutable. Verás los formatos ELF (Linux) y PE (Windows), las secciones, símbolos y strings, y el flujo general de un análisis: reconocimiento estático rápido antes de abrir un desensamblador.

⚠️ **Ética:** aplica ingeniería inversa solo a binarios propios, de práctica o cuya licencia/permiso lo autoricen. Respeta la ley y los términos de uso.

📖 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Definir** ingeniería inversa y sus casos de uso legítimos.
- 2. **Describir** la estructura de un ELF y un PE (cabeceras, secciones, símbolos).
- 3. **Aplicar** triage estático con `file`, `strings`, `nm`, `readelf`, `objdump`.
- 4. **Distintuir** análisis estático de dinámico y cuándo usar cada uno.
- 5. **Planificar** una metodología de reversing por objetivos.

📚 Temas

#	Tema	Por qué importa
1	Qué es y usos legítimos	Marco y ética
2	Formato ELF	Binarios de Linux/CTF
3	Formato PE	Binarios de Windows/malware
4	Secciones (.text/.data/.rodata)	Dónde vive código y datos
5	Símbolos y stripping	Cuánta ayuda te da el binario
6	strings y triage rápido	Primeras pistas
7	Estático vs dinámico	Estrategia de análisis
8	Metodología por objetivos	No perderse en binarios grandes

Definiciones y características

- **Ingeniería inversa:** proceso de deducir el funcionamiento de un programa a partir de su forma compilada. *Clave:* legal y valiosa en seguridad defensiva y ofensiva autorizada.
- **ELF:** formato ejecutable de Linux con cabecera, program headers (carga) y section headers (análisis). *Clave:* `readelf -h` muestra tipo, arquitectura y entry point.
- **PE:** formato de Windows (DOS header, PE header, secciones, tabla de importaciones). *Clave:* las imports revelan qué APIs usa (indicio de comportamiento).
- **Binario stripped:** sin símbolos de depuración/nombres. *Clave:* dificulta el análisis; hay que reconstruir funciones.
- **Triage estático:** primera pasada con herramientas ligeras antes de desensamblar. *Clave:* ahorra horas al orientar el análisis.

Herramientas y preparación

```
sudo apt install -y binutils file
pip install pwntools # checksec
# Para PE en Linux:
sudo apt install -y pev
```

Laboratorio guiado

Entorno propio.

1. Compila o toma un binario de práctica (`crackme`) y haz el triage:

```
bash file crackme checksec --file=crackme strings -n 6 crackme | less # busca prompts, rutas,
contraseñas, formatos
```

2. Explora la estructura ELF:

```
bash readelf -h crackme # cabecera, entry point readelf -S crackme # secciones readelf -s
crackme | head # símbolos (si no está stripped)
```

3. Localiza la función `main` y desensámbllala superficialmente:

```
bash objdump -d -M intel crackme | sed -n '/<main>:/,/ret/p' | head -40
```

4. Identifica llamadas relevantes (`strcmp` , `puts` , `scanf`) que sugieran la lógica de validación.

5. Para un PE (si dispones de uno de práctica), usa `pev` / `peframe` para listar imports y secciones:

```
bash readpe -i muestra.exe # importaciones
```

6. Escribe un mini-informe: propósito aparente, funciones clave y por dónde continuarías (estático con Ghidra o dinámico con GDB).

Ejercicios

1. Diferencia entre program headers y section headers de un ELF.
2. Extrae el entry point con `readelf` y localízalo en el desensamblado.
3. Encuentra una cadena sospechosa con `strings` y razona su función.
4. Determina si un binario está stripped y cómo lo sabes.
5. Lista tres APIs de Windows en un PE y deduce comportamiento.
6. Propón una metodología de reversing para un binario de 2 MB.

Reto verificable

Realiza el triage estático completo de un `crackme` e identifica, sin ejecutarlo, cuál es la función que compara la clave y qué API/rutina usa.

Criterio de aceptación: señalas la función de comparación (p. ej. `strcmp` /lógica propia) y justificas tu conclusión con evidencia de `strings` / `objdump` / `readelf`.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>readelf -s</code> no muestra nombres	Binario stripped; usa análisis por patrones
<code>strings</code> no revela nada útil	Cadenas cifradas/ofuscadas; pasa a dinámico
Confundir program vs section headers	Program = carga; section = análisis
<code>objdump</code> da AT&T	Añade <code>-M intel</code>
Analizar un PE con herramientas ELF	Usa <code>pev</code> / <code>readpe</code> para PE

Preguntas frecuentes

? ¿Reversing es legal? El análisis con fines de seguridad, interoperabilidad y sobre binarios propios/autorizados es legítimo; respeta licencias y jurisdicción.

? ¿Empiezo por estático o dinámico? Casi siempre triage estático primero; el dinámico confirma y desvela lo ofuscado.

? ¿Necesito los símbolos? Ayudan mucho, pero el reversing serio asume binarios stripped y reconstruye la semántica.

Referencias

- Andriesse, D. *Practical Binary Analysis*, caps. 1-2. No Starch Press.
- Eilam, E. *Reversing: Secrets of Reverse Engineering*. Wiley.
- ELF spec — <https://refspecs.linuxfoundation.org/elf/elf.pdf>
- Microsoft PE format — <https://learn.microsoft.com/windows/win32/debug/pe-format>



Material descargable

- [Guía en PDF](#) — versión imprimible de esta clase.
- [Presentación \(PPTX\)](#) — deck para proyectar en clase.



Siguiente clase

[Clase 131 - Ghidra para ingeniería inversa](#)